

CSS Popover & Position-Area

Modern Positioning for Tooltips & Dropdowns

Complete Guide: From Basics to Advanced

Easy English • Practical Examples • Copy-Ready Code

Master CSS popover API and position-area for next-generation UI positioning

Contents

1	Introduction	3
1.1	What is CSS Popover?	3
1.2	What is Position-Area?	3
1.3	Why Use Popover API?	3
1.4	Browser Support	4
2	Understanding CSS Popover	5
2.1	How Popover Works	5
2.2	Basic HTML Structure	5
2.3	Popover Attribute Values	5
2.4	JavaScript Control	5
2.5	CSS for Popover	6
3	Understanding Position-Area	7
3.1	What is Position-Area?	7
3.2	Position-Area Syntax	7
3.3	Vertical Position Values	7
3.4	Horizontal Position Values	7
3.5	Common Position-Area Combinations	7
3.6	Fallback Positions	8
4	Using the Anchor Attribute	9
4.1	What is an Anchor?	9
4.2	Setting an Anchor	9
4.3	Anchor Positioning in CSS	9
4.4	Multiple Anchors	10
5	Basic Examples	11
5.1	Example 1: Simple Tooltip	11
5.2	Example 2: Dropdown Menu	11
5.3	Example 3: Modal Dialog	12
5.4	Example 4: Info Popover	13
6	Advanced Positioning Techniques	15
6.1	Inset Adjustment	15
6.2	Multiple Position Options	15
6.3	Responsive Positioning	15
6.4	Smart Positioning with JavaScript	16

6.5	Animation Variants by Position	16
7	Real-World Examples	18
7.1	Example 1: User Profile Popover	18
7.2	Example 2: Feature Highlight Tooltip	19
7.3	Example 3: Notification Center	20
7.4	Example 4: Filter Panel	21
8	Accessibility Features	23
8.1	Built-in Accessibility	23
8.2	Focus Management	23
8.3	ARIA Attributes	23
8.4	Keyboard Navigation	24
8.5	Screen Reader Testing	24
9	JavaScript Integration	25
9.1	Showing and Hiding	25
9.2	Event Listeners	25
9.3	Positioning with JavaScript	26
9.4	Animation Control	26
10	Common Issues & Solutions	27
10.1	Issue 1: Popover Not Displaying	27
10.2	Issue 2: Position Not Working	27
10.3	Issue 3: Content Hidden Behind Backdrop	27
10.4	Issue 4: Browser Support Fallback	28
10.5	Issue 5: Flickering on Resize	28
11	Browser Compatibility	29
11.1	Current Support Matrix	29
11.2	Feature Detection	29
11.3	Progressive Enhancement	29
12	Copy-Ready Code Templates	31
12.1	Template 1: Basic Tooltip	31
12.2	Template 2: Dropdown Menu	31
12.3	Template 3: Modal Dialog	32
12.4	Template 4: Complete Component	33
13	Best Practices & Guidelines	35
13.1	Do This	35
13.2	Don't Do This	35
13.3	Performance Tips	36
13.4	Mobile Considerations	36
14	Summary & Key Takeaways	37
14.1	What You've Learned	37
14.2	Quick Reference	37
14.3	When to Use Popover API	37

14.4 Next Steps	38
---------------------------	----

Chapter 1

Introduction

1.1 What is CSS Popover?

The CSS Popover API is a modern web standard that simplifies creating floating UI elements like tooltips, dropdowns, and popovers. It handles positioning, layering, and accessibility automatically—without needing complex JavaScript.

Definition

The Popover API provides a built-in way to display content on top of other content. It automatically manages z-index, focus, and dismissal behavior. The `position-area` property controls where the popover appears relative to its anchor.

1.2 What is Position-Area?

The `position-area` property specifies where a popover should appear relative to its anchor element. It's part of the Anchor Positioning specification and gives you fine-grained control over placement.

1.3 Why Use Popover API?

- **Simplicity:** No complex JavaScript or positioning libraries needed
- **Accessibility:** Built-in keyboard and screen reader support
- **Performance:** Optimized browser rendering
- **Consistency:** Works consistently across modern browsers
- **Automatic Positioning:** Handles overflow and edge cases
- **Stack Management:** Automatic z-index handling

1.4 Browser Support

Browser	Support Status
Chrome 114+	Full Support
Edge 114+	Full Support
Safari 17+	Full Support
Firefox 125+	Full Support
Opera 100+	Full Support

Chapter 2

Understanding CSS Popover

2.1 How Popover Works

The Popover API provides three key components:

1. **Popover Element:** The content you want to show/hide
2. **Trigger Button:** What activates the popover
3. **Anchor:** The reference element for positioning

2.2 Basic HTML Structure

```
<!-- Trigger button -->
<button popovertarget="my-popover">
  Click Me
</button>

<!-- Popover element -->
<div id="my-popover" popover="auto">
  This is a popover!
</div>
```

2.3 Popover Attribute Values

Value	Behavior
auto	Toggle on click, close with Escape key
manual	Only show/hide with JavaScript

2.4 JavaScript Control

```
const popover = document.getElementById('my-popover');

// Show the popover
popover.showPopover();
```

```
// Hide the popover
popover.hidePopover();

// Toggle popover
popover.togglePopover();

// Check if visible
if (popover.matches(':popover-open')) {
  console.log('Popover is open!');
}
```

2.5 CSS for Popover

```
[popover] {
  /* Default styling */
  background: white;
  border: 1px solid #ddd;
  border-radius: 8px;
  padding: 16px;
  box-shadow: 0 4px 12px rgba(0, 0, 0, 0.15);
}

/* Style when popover is open */
[popover]:popover-open {
  animation: slideIn 0.3s ease-out;
}

@keyframes slideIn {
  from {
    opacity: 0;
    transform: translateY(-10px);
  }
  to {
    opacity: 1;
    transform: translateY(0);
  }
}
```


Chapter 3

Understanding Position-Area

3.1 What is Position-Area?

The `position-area` property defines where an anchored element (like a popover) should appear relative to its anchor element. It uses a two-part syntax: vertical alignment and horizontal alignment.

3.2 Position-Area Syntax

```
[popover] {  
  position-area: top center;  
  /* Format: <vertical> <horizontal> */  
}
```

3.3 Vertical Position Values

Value	Position
<code>top</code>	Above the anchor
<code>bottom</code>	Below the anchor
<code>center</code>	Vertically centered
<code>start</code>	At the anchor's start
<code>end</code>	At the anchor's end

3.4 Horizontal Position Values

Value	Position
<code>left</code>	Left of the anchor
<code>right</code>	Right of the anchor
<code>center</code>	Horizontally centered
<code>start</code>	At the anchor's start
<code>end</code>	At the anchor's end

3.5 Common Position-Area Combinations

```
/* Below and centered */
[popover] {
  position-area: bottom center;
}

/* Above and left-aligned */
[popover] {
  position-area: top left;
}

/* Right side, vertically centered */
[popover] {
  position-area: center right;
}

/* Below, on the right */
[popover] {
  position-area: bottom right;
}

/* Centered both ways */
[popover] {
  position-area: center center;
}
```

3.6 Fallback Positions

Handle cases where primary position isn't available:

```
[popover] {
  position-area: top center / bottom center;
  /* Primary position: top center */
  /* Fallback: bottom center if no space */
}
```

Chapter 4

Using the Anchor Attribute

4.1 What is an Anchor?

An anchor is the element that a popover positions itself relative to. By default, it's the trigger button, but you can specify any element.

4.2 Setting an Anchor

```
<!-- Anchor element -->
<button id="trigger">Show Menu</button>

<!-- Popover with explicit anchor -->
<div id="dropdown" popover="auto" anchor="trigger">
  <ul>
    <li>Option 1</li>
    <li>Option 2</li>
  </ul>
</div>
```

4.3 Anchor Positioning in CSS

```
[popover] {
  /* Anchor the popover */
  anchor-name: --my-anchor;

  /* Position relative to anchor */
  position-area: bottom center;
}

/* Element to anchor to */
#trigger {
  anchor-name: --my-anchor;
}
```

4.4 Multiple Anchors

```
/* Define anchor point */
.menu-button {
  --anchor-point: bottom;
}

/* Use anchor in popover */
[popover] {
  position-area: var(--anchor-point) center;
}
```

Chapter 5

Basic Examples

5.1 Example 1: Simple Tooltip

```
<button popovertarget="tooltip">Hover</button>

<div id="tooltip" popover="auto">
  This is helpful information!
</div>
```

```
[popover] {
  position-area: top center;
  background: #333;
  color: white;
  padding: 8px 12px;
  border-radius: 4px;
  font-size: 12px;
  width: max-content;
}

[popover]:popover-open {
  animation: fadeIn 0.2s ease-out;
}

@keyframes fadeIn {
  from { opacity: 0; }
  to { opacity: 1; }
}
```

5.2 Example 2: Dropdown Menu

```
<button popovertarget="menu" id="menu-btn">
  Menu
</button>

<div id="menu" popover="auto" anchor="menu-btn">
  <ul>
    <li><a href="#">Profile</a></li>
```

```

        <li><a href="#">Settings</a></li>
        <li><a href="#">Logout</a></li>
    </ul>
</div>

```

```

[popover] {
    position-area: bottom left;
    background: white;
    border: 1px solid #ddd;
    border-radius: 8px;
    padding: 0;
    min-width: 200px;
    box-shadow: 0 4px 12px rgba(0,0,0,0.1);
}

[popover] ul {
    list-style: none;
    margin: 0;
    padding: 0;
}

[popover] li {
    border-bottom: 1px solid #f0f0f0;
}

[popover] li:last-child {
    border-bottom: none;
}

[popover] a {
    display: block;
    padding: 12px 16px;
    text-decoration: none;
    color: #333;
    transition: background 0.2s;
}

[popover] a:hover {
    background: #f5f5f5;
}

```

5.3 Example 3: Modal Dialog

```

<button popovertarget="dialog">Open Dialog</button>

<div id="dialog" popover="auto">
    <h2>Confirm Action</h2>
    <p>Are you sure?</p>
    <button onclick="closeDialog()">Cancel</button>
    <button>Confirm</button>
</div>

```

```
[popover] {
  position-area: center center;
  background: white;
  border-radius: 12px;
  padding: 24px;
  width: 90%;
  max-width: 400px;
  box-shadow: 0 20px 60px rgba(0,0,0,0.3);
}

[popover]::backdrop {
  background: rgba(0, 0, 0, 0.5);
}

[popover] h2 {
  margin: 0 0 16px 0;
}

[popover] button {
  margin-top: 16px;
  margin-right: 8px;
}
```

5.4 Example 4: Info Popover

```
<span id="info-trigger">?</span>

<div id="info" popover="auto" anchor="info-trigger">
  <p>Useful information goes here.</p>
</div>
```

```
#info-trigger {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  width: 20px;
  height: 20px;
  background: #667eea;
  color: white;
  border-radius: 50%;
  cursor: help;
  font-size: 12px;
  font-weight: bold;
}

[popover] {
  position-area: top center;
  background: white;
  border: 2px solid #667eea;
  border-radius: 8px;
  padding: 12px;
```

```
max-width: 250px;  
font-size: 13px;  
}
```


Chapter 6

Advanced Positioning Techniques

6.1 Inset Adjustment

Fine-tune popover position with inset:

```
[popover] {  
  position-area: bottom center;  
  inset: 8px auto auto auto;  
  /* Add 8px gap below anchor */  
}
```

6.2 Multiple Position Options

Provide fallback positions:

```
[popover] {  
  position-area:  
    top center /  
    bottom center /  
    right center;  
  /* Try top first, then bottom, then right */  
}
```

6.3 Responsive Positioning

Adjust positions based on screen size:

```
[popover] {  
  position-area: top center;  
}  
  
@media (max-width: 768px) {  
  [popover] {  
    position-area: bottom center;  
    inset: 12px auto auto auto;  
  }  
}
```

```
@media (max-width: 480px) {
  [popover] {
    position-area: bottom center;
    width: 100%;
    max-width: none;
  }
}
```

6.4 Smart Positioning with JavaScript

```
const popover = document.getElementById('my-popover');
const trigger = document.getElementById('trigger');

function updatePosition() {
  const rect = trigger.getBoundingClientRect();
  const spaceBelow = window.innerHeight - rect.bottom;
  const spaceAbove = rect.top;

  if (spaceAbove > 300) {
    popover.style.positionArea = 'top center';
  } else if (spaceBelow > 300) {
    popover.style.positionArea = 'bottom center';
  } else {
    popover.style.positionArea = 'center center';
  }
}

trigger.addEventListener('click', updatePosition);
window.addEventListener('resize', updatePosition);
```

6.5 Animation Variants by Position

```
[popover]:popover-open {
  animation: slideIn 0.3s ease-out;
}

[popover][style*="top"]::popover-open {
  animation: slideDownIn 0.3s ease-out;
}

[popover][style*="bottom"]::popover-open {
  animation: slideUpIn 0.3s ease-out;
}

[popover][style*="left"]::popover-open {
  animation: slideRightIn 0.3s ease-out;
}

@keyframes slideDownIn {
  from { opacity: 0; transform: translateY(-10px); }
```

```
}  
  
@keyframes slideUpIn {  
  from { opacity: 0; transform: translateY(10px); }  
}  
  
@keyframes slideRightIn {  
  from { opacity: 0; transform: translateX(-10px); }  
}
```

Chapter 7

Real-World Examples

7.1 Example 1: User Profile Popover

```
<button id="profile-btn" popovertarget="profile-pop">
  John Doe
</button>

<div id="profile-pop" popover="auto" anchor="profile-btn">
  <div class="profile-header">
    
    <h3>John Doe</h3>
  </div>
  <ul class="profile-menu">
    <li><a href="#">My Profile</a></li>
    <li><a href="#">Settings</a></li>
    <li><a href="#">Help</a></li>
    <li><a href="#">Sign Out</a></li>
  </ul>
</div>
```

```
#profile-pop {
  position-area: bottom right;
  background: white;
  border-radius: 12px;
  width: 280px;
  box-shadow: 0 8px 24px rgba(0,0,0,0.12);
  padding: 0;
}

.profile-header {
  padding: 16px;
  border-bottom: 1px solid #f0f0f0;
  display: flex;
  align-items: center;
  gap: 12px;
}

.profile-header img {
  width: 40px;
```

```
    height: 40px;
    border-radius: 50%;
}

.profile-menu {
    list-style: none;
    padding: 0;
    margin: 0;
}

.profile-menu a {
    display: block;
    padding: 12px 16px;
    text-decoration: none;
    color: #333;
}

.profile-menu a:hover {
    background: #f5f5f5;
}
```

7.2 Example 2: Feature Highlight Tooltip

```
.feature-hint {
    position-area: top center;
    background: linear-gradient(135deg, #667eea, #764ba2);
    color: white;
    padding: 12px 16px;
    border-radius: 8px;
    font-size: 13px;
    max-width: 200px;
    box-shadow: 0 4px 12px rgba(102, 126, 234, 0.3);
}

.feature-hint::before {
    content: '';
    margin-right: 8px;
}

.feature-hint:popover-open {
    animation: bounceIn 0.4s cubic-bezier(0.68, -0.55, 0.265, 1.55);
}

@keyframes bounceIn {
    0% {
        opacity: 0;
        transform: translateY(10px) scale(0.9);
    }
    50% {
        transform: scale(1.05);
    }
    100% {
```

```

        opacity: 1;
        transform: translateY(0) scale(1);
    }
}

```

7.3 Example 3: Notification Center

```

<button id="notify-btn" popovertarget="notify-center">
  <span class="badge">3</span>
</button>

<div id="notify-center" popover="auto" anchor="notify-btn">
  <div class="notifications">
    <div class="notification">
      <strong>New message</strong>
      <p>You have a new message from Sarah</p>
    </div>
    <div class="notification">
      <strong>Update complete</strong>
      <p>Your profile has been updated</p>
    </div>
  </div>
</div>

```

```

#notify-center {
  position-area: bottom right;
  background: white;
  border-radius: 12px;
  width: 320px;
  max-height: 400px;
  overflow-y: auto;
  box-shadow: 0 8px 24px rgba(0,0,0,0.15);
}

.notification {
  padding: 16px;
  border-bottom: 1px solid #f0f0f0;
  transition: background 0.2s;
}

.notification:hover {
  background: #f9f9f9;
}

.notification:last-child {
  border-bottom: none;
}

.badge {
  background: #ff6b6b;
  color: white;
  border-radius: 50%;
}

```

```
padding: 2px 6px;
font-size: 12px;
font-weight: bold;
}
```

7.4 Example 4: Filter Panel

```
<button id="filter-btn" popovertarget="filters">
  Filters
</button>

<div id="filters" popover="auto" anchor="filter-btn">
  <div class="filter-section">
    <label>Price Range</label>
    <input type="range" min="0" max="1000">
  </div>
  <div class="filter-section">
    <label>Category</label>
    <select>
      <option>Electronics</option>
      <option>Clothing</option>
    </select>
  </div>
  <div class="filter-buttons">
    <button>Apply</button>
    <button>Reset</button>
  </div>
</div>
```

```
#filters {
  position-area: bottom left;
  background: white;
  border: 1px solid #ddd;
  border-radius: 8px;
  padding: 16px;
  width: 250px;
  box-shadow: 0 4px 12px rgba(0,0,0,0.1);
}

.filter-section {
  margin-bottom: 16px;
}

.filter-section:last-of-type {
  margin-bottom: 0;
}

.filter-section label {
  display: block;
  font-weight: 600;
  margin-bottom: 8px;
  font-size: 13px;
}
```

```
}  
  
.filter-buttons {  
  display: flex;  
  gap: 8px;  
  margin-top: 16px;  
  padding-top: 16px;  
  border-top: 1px solid #f0f0f0;  
}  
  
.filter-buttons button {  
  flex: 1;  
  padding: 8px;  
  border-radius: 4px;  
}
```


Chapter 8

Accessibility Features

8.1 Built-in Accessibility

Popover API includes automatic accessibility:

- Escape key closes popover
- Focus automatically managed
- Screen reader support
- Backdrop prevents outside interaction
- ARIA attributes automatically set

8.2 Focus Management

Ensure proper focus handling:

```
const popover = document.getElementById('my-popover');

popover.addEventListener('toggle', (e) => {
  if (e.newState === 'open') {
    // Focus first interactive element
    const firstButton = popover.querySelector('button');
    if (firstButton) firstButton.focus();
  }
});
```

8.3 ARIA Attributes

Add semantic markup:

```
<button
  popovertarget="menu"
  aria-label="Open menu"
  aria-expanded="false">
  Menu
```

```
</button>

<div
  id="menu"
  popover="auto"
  role="menu"
  aria-label="Navigation menu">
  <!-- Content -->
</div>
```

8.4 Keyboard Navigation

```
const popover = document.getElementById('my-popover');
const buttons = popover.querySelectorAll('button');

popover.addEventListener('keydown', (e) => {
  if (e.key === 'Tab') {
    // Handle tab navigation within popover
    const firstBtn = buttons[0];
    const lastBtn = buttons[buttons.length - 1];

    if (e.shiftKey && document.activeElement === firstBtn) {
      e.preventDefault();
      lastBtn.focus();
    } else if (!e.shiftKey && document.activeElement === lastBtn) {
      e.preventDefault();
      firstBtn.focus();
    }
  }
});
```

8.5 Screen Reader Testing

Test with screen readers:

- NVDA (Windows)
- JAWS (Windows)
- VoiceOver (macOS/iOS)
- TalkBack (Android)

Chapter 9

JavaScript Integration

9.1 Showing and Hiding

```
const popover = document.getElementById('my-popover');

// Show popover
popover.showPopover();

// Hide popover
popover.hidePopover();

// Toggle popover
popover.togglePopover();

// Check if open
const isOpen = popover.matches(':popover-open');
console.log(isOpen);
```

9.2 Event Listeners

```
const popover = document.getElementById('my-popover');

// When state changes
popover.addEventListener('toggle', (event) => {
  if (event.newState === 'open') {
    console.log('Popover opened');
  } else {
    console.log('Popover closed');
  }
});

// Close on outside click
document.addEventListener('click', (e) => {
  if (!popover.contains(e.target)) {
    popover.hidePopover();
  }
});
```

9.3 Positioning with JavaScript

```
function positionPopover() {
  const popover = document.getElementById('my-popover');
  const trigger = document.getElementById('trigger');
  const rect = trigger.getBoundingClientRect();

  // Calculate available space
  const spaceBelow = window.innerHeight - rect.bottom;
  const spaceAbove = rect.top;

  if (spaceBelow > 300) {
    popover.style.positionArea = 'bottom center';
  } else if (spaceAbove > 300) {
    popover.style.positionArea = 'top center';
  }
}

document.getElementById('trigger')
  .addEventListener('click', positionPopover);
```

9.4 Animation Control

```
const popover = document.getElementById('my-popover');

popover.addEventListener('toggle', (e) => {
  const content = popover.querySelector('.content');

  if (e.newState === 'open') {
    content.classList.add('animate-in');
    content.classList.remove('animate-out');
  } else {
    content.classList.add('animate-out');
    content.classList.remove('animate-in');
  }
});
```

Chapter 10

Common Issues & Solutions

10.1 Issue 1: Popover Not Displaying

Problem: Popover element doesn't show.

Solution: Ensure proper setup:

```
<!-- Correct -->
<button popovertarget="my-pop">Click</button>
<div id="my-pop" popover="auto">Content</div>

<!-- Check: IDs must match -->
<!-- Check: Element must have popover attribute -->
```

10.2 Issue 2: Position Not Working

Problem: position-area doesn't apply.

Solution: Use valid values and check browser support:

```
/* Valid */
[popover] {
  position-area: top center;
}

/* Invalid - typo */
[popover] {
  position-area: tp center; /* Wrong! */
}
```

10.3 Issue 3: Content Hidden Behind Backdrop

Problem: Popover appears behind other elements.

Solution: Popover API handles z-index automatically. If issues persist:

```
[popover] {
  z-index: 1000;
  /* Usually not needed - Popover API handles this */
}
```

10.4 Issue 4: Browser Support Fallback

Problem: Older browsers don't support Popover API.

Solution: Implement polyfill or fallback:

```
if (!HTMLElement.prototype.showPopover) {  
  // Fallback: use custom solution  
  document.documentElement.classList.add('no-popover-api');  
}  
  
/* CSS fallback */  
@supports not (position-area: top center) {  
  [popover] {  
    position: fixed;  
    top: 50%;  
    left: 50%;  
    transform: translate(-50%, -50%);  
  }  
}
```

10.5 Issue 5: Flickering on Resize

Problem: Popover flickers when window resizes.

Solution: Debounce resize handler:

```
let resizeTimeout;  
  
window.addEventListener('resize', () => {  
  clearTimeout(resizeTimeout);  
  resizeTimeout = setTimeout(() => {  
    // Update popover position  
  }, 150);  
});
```

Chapter 11

Browser Compatibility

11.1 Current Support Matrix

Feature	Support Level	Since
Popover API	Chrome 114+, Safari 17+	2024
position-area	Chrome 114+, Safari 17+	2024
Anchor Positioning	Chrome 125+, Safari 18+	2024
Firefox Support	Firefox 125+	2024

11.2 Feature Detection

```
// Check for Popover API support
function hasPopoverSupport() {
    return HTMLElement.prototype.hasOwnProperty('popover');
}

// Check for position-area support
function hasPositionAreaSupport() {
    const element = document.createElement('div');
    element.style.positionArea = 'top center';
    return element.style.positionArea !== '';
}

if (!hasPopoverSupport()) {
    console.log('Popover API not supported');
    // Use fallback
}
```

11.3 Progressive Enhancement

```
<!-- Popover (modern) -->
<div popover="auto" id="modern-menu">
    Modern popover content
</div>

<!-- Fallback (traditional) -->
```

```
<div id="legacy-menu" class="dropdown" style="display:none">
  Fallback menu content
</div>

<script>
if (!HTMLElement.prototype.showPopover) {
  document.getElementById('modern-menu').hidden = true;
  document.getElementById('legacy-menu').style.display = 'block';
}
</script>
```


Chapter 12

Copy-Ready Code Templates

12.1 Template 1: Basic Tooltip

```
<button id="tooltip-btn" popovertarget="tooltip">Help</button>
<div id="tooltip" popover="auto">Helpful information</div>
```

```
#tooltip {
  position-area: top center;
  background: #333;
  color: white;
  padding: 8px 12px;
  border-radius: 4px;
  font-size: 12px;
  width: max-content;
  box-shadow: 0 2px 8px rgba(0,0,0,0.2);
}

#tooltip:popover-open {
  animation: fadeIn 0.2s ease-out;
}

@keyframes fadeIn {
  from { opacity: 0; }
  to { opacity: 1; }
}
```

12.2 Template 2: Dropdown Menu

```
<button id="menu-btn" popovertarget="menu">Menu</button>
<div id="menu" popover="auto" anchor="menu-btn">
  <ul>
    <li><a href="#">Profile</a></li>
    <li><a href="#">Settings</a></li>
    <li><a href="#">Logout</a></li>
  </ul>
</div>
```

```
#menu {
  position-area: bottom left;
  background: white;
  border: 1px solid #ddd;
  border-radius: 8px;
  padding: 0;
  min-width: 200px;
  box-shadow: 0 4px 12px rgba(0,0,0,0.1);
}

#menu ul {
  list-style: none;
  margin: 0;
  padding: 0;
}

#menu li {
  border-bottom: 1px solid #f0f0f0;
}

#menu a {
  display: block;
  padding: 12px 16px;
  color: #333;
  text-decoration: none;
  transition: background 0.2s;
}

#menu a:hover {
  background: #f5f5f5;
}
```

12.3 Template 3: Modal Dialog

```
<button popovertarget="dialog">Open</button>
<div id="dialog" popover="auto">
  <h2>Confirm</h2>
  <p>Continue?</p>
  <button onclick="document.getElementById('dialog').hidePopover()">
    Cancel
  </button>
  <button>Confirm</button>
</div>
```

```
#dialog {
  position-area: center center;
  background: white;
  border-radius: 12px;
  padding: 24px;
  width: 90%;
  max-width: 400px;
```

```

    box-shadow: 0 20px 60px rgba(0,0,0,0.3);
}

#dialog::backdrop {
    background: rgba(0, 0, 0, 0.5);
}

#dialog:popover-open {
    animation: scaleIn 0.3s ease-out;
}

@keyframes scaleIn {
    from { opacity: 0; transform: scale(0.95); }
    to { opacity: 1; transform: scale(1); }
}

```

12.4 Template 4: Complete Component

```

<div class="popover-container">
  <button class="popover-trigger" popovertarget="popup">
    Open Popover
  </button>

  <div id="popup" popover="auto" class="popover-content">
    <div class="popover-header">
      <h3>Popover Title</h3>
    </div>
    <div class="popover-body">
      <p>Popover content goes here.</p>
    </div>
    <div class="popover-footer">
      <button onclick="document.getElementById('popup').hidePopover()">
        Close
      </button>
    </div>
  </div>
</div>

```

```

.popover-trigger {
  padding: 10px 16px;
  background: #667eea;
  color: white;
  border: none;
  border-radius: 6px;
  cursor: pointer;
  font-size: 14px;
}

.popover-content {
  position-area: bottom center;
  background: white;
  border-radius: 8px;
}

```

```
    min-width: 300px;
    box-shadow: 0 4px 12px rgba(0,0,0,0.15);
    padding: 0;
}

.popover-header {
    padding: 16px;
    border-bottom: 1px solid #f0f0f0;
}

.popover-body {
    padding: 16px;
}

.popover-footer {
    padding: 12px 16px;
    border-top: 1px solid #f0f0f0;
    text-align: right;
}

.popover-content:popover-open {
    animation: slideUp 0.3s ease-out;
}

@keyframes slideUp {
    from { opacity: 0; transform: translateY(10px); }
    to { opacity: 1; transform: translateY(0); }
}
```

Chapter 13

Best Practices & Guidelines

13.1 Do This

- Use semantic HTML with proper IDs
- Include escape key support (auto)
- Add focus management for accessibility
- Test across multiple browsers
- Provide visual feedback
- Use meaningful position-area values
- Handle mobile responsiveness
- Test with keyboard navigation

13.2 Don't Do This

- Don't nest popovers without careful handling
- Don't ignore focus management
- Don't assume all browsers support Popover API
- Don't use popovers for critical information only
- Don't make popovers difficult to dismiss
- Don't block user interaction without reason
- Don't ignore mobile considerations
- Don't forget fallback content

13.3 Performance Tips

```
/* Use will-change for smooth animations */
[popover]:popover-open {
  will-change: transform, opacity;
  animation: slideIn 0.3s ease-out;
}

[popover] {
  contain: layout style paint;
  /* Isolate from layout recalculations */
}
```

13.4 Mobile Considerations

```
@media (max-width: 640px) {
  [popover] {
    position-area: bottom center;
    width: 95%;
    max-height: 60vh;
    overflow-y: auto;
  }

  [popover]::backdrop {
    background: rgba(0, 0, 0, 0.7);
  }
}
```

Chapter 14

Summary & Key Takeaways

14.1 What You've Learned

- Popover API basics and HTML structure
- position-area for precise positioning
- Anchor elements and positioning control
- Real-world examples (tooltips, menus, dialogs)
- Accessibility features and best practices
- JavaScript integration and event handling
- Browser compatibility and fallbacks
- Copy-ready templates for common use cases

14.2 Quick Reference

Concept	Key Points
Popover	HTML popover="auto manual" attribute
Trigger	Button with popovertarget attribute
position-area	<vertical> <horizontal> positioning
Methods	showPopover(), hidePopover(), togglePopover()
Events	toggle event with newState
Accessibility	Automatic Escape key, focus management

14.3 When to Use Popover API

Perfect for:

- Dropdowns and context menus
- Tooltips and help text
- Modal dialogs

- Floating panels
- Info popovers

Consider alternatives for:

- Complex nested components
- Older browser support needed
- Animated, complex layouts

14.4 Next Steps

1. Implement a simple tooltip with Popover API
2. Test position-area values for your use case
3. Add keyboard navigation support
4. Test across browsers and devices
5. Gather user feedback
6. Refine and iterate

Master Modern Web Positioning!

Popover API simplifies building interactive, accessible UI components.